



University of Deusto

Bachelor in Data Science and Artificial Intelligence

INTEGRATION OF VISION LANGUAGE MODELS (VLMS) IN A
MULTI-AGENT REINFORCEMENT LEARNING (MARL)
ENVIRONMENT FOR COOPERATIVE ROBOTIC TASKS IN NVIDIA
ISAAC SIM

Student Unai Olaizola Osa

DNI 73032586L

Email unai.olaizola.osa@opendeusto.es

GitHub repository [VLMS IsaacSim](#)

April 17, 2026

Acknowledgments

Abstract

This project proposes the development and implementation of a Multi-Agent Reinforcement Learning (MARL) system integrated with Vision Language Models (VLMs) for the execution of cooperative robotic tasks within a simulated NVIDIA Isaac Sim environment.

One of the major challenges lies in translating commands from a natural language format into precise robotic actions. To address this, the reasoning capabilities of the VLMs have been exploited for complex instruction interpretation and decision-making. A controlled environment where robotic agents must cooperate to manipulate and respond to objects in order to achieve a shared goal has been designed and shaped for Reinforcement Learning training using NVIDIA's Isaac Lab framework. The integration of these models has enhanced generalization and promote a more cooperative behavior of the agents against variations in the scene. The learning efficiency has been evaluated in addition to metrics such as the stability of the policies and the transferable capacity of the model(s). Additionally, the project includes an in-depth analysis of state-of-the-art (SOTA) models, providing a comprehensive review of the current methodologies and core concepts that form the theoretical foundation of the proposed architecture, presented in the documentation.

The expected outcome is a demonstration and reflection of the empowerment of multi-modal models within robotic environments, which can be applied to real-world problems.

Index terms

Vision language models, multi-agent reinforcement learning system, computer vision

Contents

Acknowledgments	i
1 Introduction	1
1.1 Problem context	1
1.2 Motivation	1
1.3 Main objectives and specific objectives	1
1.4 Project scope and limitations	1
2 Theoretical foundations	2
2.1 Vision Language Models	2
2.1.1 History of VLMs	2
2.1.2 Elements of a VLMs	7
2.2 Reinforcement Learning	10
2.3 Multi-agent systems	10
2.4 State of the Art	10
2.4.1 VLMs evolution	10
2.4.2 Current models	10
2.4.3 Benchmarks	10
2.4.4 VLMs in robotics	10
2.4.5 Frameworks	10
3 Solution approach	11
3.1 Solution proposal	11
3.2 System architecture	11
3.3 Tools used	11
3.4 Workflow	11

4	Solution development	12
4.1	IsaacSim environment definition	12
4.1.1	Agents	12
4.1.2	Cameras	12
4.1.3	Physics	12
4.2	Individual component operation	12
5	Experimentation and evolution	13
5.1	First scenarios	13
5.2	Last scenario	13
5.3	Successful and failure tries	13
5.4	Results obtained in each scenario	13
5.5	Result discussion	13
6	Conclusions	14
6.1	Conclusions and lessons learned	14
6.2	Limitations found	14
6.3	Future work	15
7	Appendix	17
7.1	Instalation guide	17
7.2	Relevant code	17
7.3	Prompts used	17
7.4	Result table	17
7.5	Tensorboard analysis	17

List of Figures

2.1	Graphical example of phrase grounding	4
2.2	CLIP workflow	6
2.3	Vision transformer architecture	9

List of Pseudocodes

1. INTRODUCTION

1.1 PROBLEM CONTEXT

1.2 MOTIVATION

1.3 MAIN OBJECTIVES AND SPECIFIC OBJECTIVES

1.4 PROJECT SCOPE AND LIMITATIONS

2. THEORETICAL FOUNDATIONS

2.1 VISION LANGUAGE MODELS

Vision language models (VLMs) are artificial intelligence (AI) models that combine both computer vision (CV) and natural language processing (NLP) techniques and capabilities, these models are designed to jointly understand and reason visual and textual information simultaneously [1] [3]. Among the most well-known models which will be covered when corresponded, models such as GPT-5, Gemini and LLaVA could be highlighted.

In comparison to the traditional models that process data modalities independently, VLMs have been shaped to align textual and visual data, enabling to reason complex scenes and natural language instructions.

2.1.1 History of VLMs

VLMs are the result of the evolution to earlier image captioning systems, systems which were designed primarily to take isolated images and produce descriptions. However, this systems just received the image as input, not an image-text pair input as the vision language models do now [8].

2.1.1.1 Earliest models

The early image captioning systems from around year 2010 used the *encoder-decoder* architecture, which will be described later on. These models would encode the patched images and vectorize them into feature vectors, which were then fed to the decoder part in order to generate the associated output description. When it comes to the strategies implemented by this earliest models include combination of handcrafted *visual features* to encode the image patches and *n-gram* templates to generate the description.

Those visual features can refer to any visual property like points, edges or even objects in the image.

On the other hand, n-grams are statistical language models that calculate the probability of the next word in a sequence from a fixed size window of previous words. Depending on the number of previous words considered, we can get bigram models if just one previous word is considered, if it's two is a trigram model, ... up to $n - 1$ considered as *n-gram* models and finally an unigram if no previous context is considered.

The general approximation for a sequence of words $w_1, \dots, w_m$ is:

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i \mid w_{i-(n-1)}, \dots, w_{i-1})$$

- Unigram Model ($n = 1$): No context, independent probabilities.

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i)$$

- Bigram Model ($n = 2$): Depends on the single previous word.

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i \mid w_{i-1})$$

- Trigram Model ($n = 3$): Depends on the two previous words.

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i \mid w_{i-2}, w_{i-1})$$

2.1.1.2 Deep learning based models

When the deep learning neural networks arose and became dominant near year 2015, newer models and strategies also emerged. Convolutional neural networks (CNNs) started to being used for image encoding and recurrent neural networks (RNNs) to generate the captions.

However by 2018, the transformer architecture replaced the sequential recurrent neural networks in the role as decoders. In parallel, the network parameter training started relying on image-text pair datasets, and the scope of VLMs widened until reaching other tasks. Among those tasks, *phrase grounding* could be mentioned. Task involving both natural language processing and computer vision where words or phrases within a sentence are associated to corresponding patches in an image [5]. Consists of identifying the spatial location within an image that aligns with the meaning of a given phrase, enabling models to map visual and textual data.

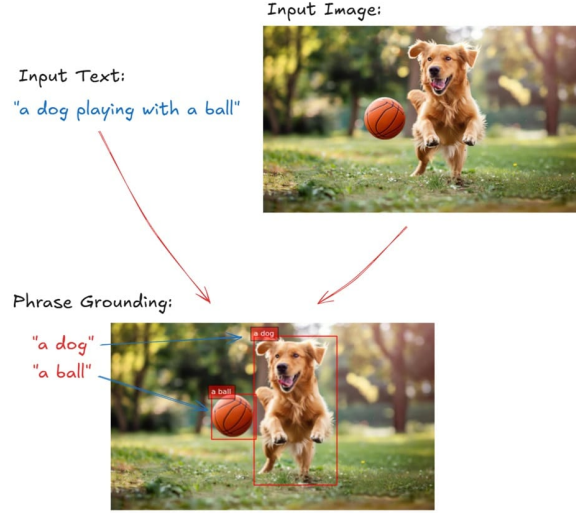


Figure 2.1: Graphical example of phrase grounding

Another task which still remains as perhaps the most relevant one is *visual question answering* (VQA). Specific task where the model is provided with an image and a specific related question, and it must generate an accurate response to it based on the visual content. Being its core pillars the object detection duty for identifying the objects in the image, then scene understanding to map the relations between objects and then finally text comprehension for embedded text interpretation within an image [2].

2.1.1.3 2021 - Release of CLIP

In the year 2021, OpenAI launched the approach that would totally revolution the evolution of vision language models, they released the method of *contrastive language-image pretraining* also known as *CLIP*. Technique for training an image-text pair into neural network models, having an exclusive neural network for image and text understanding purposes, adopting a contrastive objective.

When it comes to the algorithm itself, each neural network receives the input in the corresponding format and returns a single semantic/visual representative vector [7]. The objective to maximize by both models is the best alignment between both output vectors in the shared vector space, mapping the similar text-image pairs as close as possible while putting apart the most dissimilar ones. The training relies on a huge dataset with image-text pairs as and the two output vectors are considered similar depending on the *dot product* value.

$$s_{i,j} = \mathbf{v}_i \cdot \mathbf{w}_j = \sum_{k=1}^d v_{i,k} w_{j,k} \quad (2.1)$$

where $\mathbf{v}_i$ and $\mathbf{w}_j$ represent the embedding vectors generated by the text and image encoders respectively, and d denotes the dimensionality of the vector space. The loss function used is the multi-class N-pair loss, which is a symmetric *cross-entropy loss* over the similarity scores. The function fosters the dot product between the matching image and text vectors to be as high as possible, while discouraging high dot products between mismatching pairs.

$$\mathcal{L} = -\frac{1}{N} \sum_i \ln \frac{e^{v_i \cdot w_i / \tau}}{\sum_j e^{v_i \cdot w_j / \tau}} - \frac{1}{N} \sum_j \ln \frac{e^{v_j \cdot w_j / \tau}}{\sum_i e^{v_i \cdot w_j / \tau}} \quad (2.2)$$

being parameter T the temperature parameter ($T > 0$) which is parametrized and learned during the training. Other loss functions are possible, such as the *Sigmoid CLIP* function.

Once gone over the algorithm's properties, let's dive deep into the architecture:

- Image model: when it comes to the image encoder used in most of the CLIP versions, they all usually are some kind of *vision transformers* (ViT). Vision transformers often vary on the patch size, being that size the $n \times n$ dimensions of the patch the image was divided into. Alternatively, *convolutional neural networks* (CNNs) can be used as well (such as *ResNet*), the essential thing is to ensure the sizes of the image and text model output vectors are the same size.
- Text model: the text encoding models are usually *transformers*. Even if it is heavily inspired on the transformers report published by OpenAI [4] and shares similarities with the *GPT* architecture (decoder-only), the text model serves as a text encoder. Furthermore, its functionality is inspired by encoder only models such as *BERT*, while sharing the dimensionality and number of parameters with decoder-only models as mentioned before.

After going over the properties of both encoders that make the whole CLIP model, I will detail the model's workflow inspired on the image below 2.2.

1. The image of the image-text pair input is first splitted into smaller sized images named patches and then fed into the vision encoder, which as mentioned earlier, can be any kind of architecture such as a vision transformer (ViT) or convolutional neural network (CNN). The output of the vision encoder will in fact be a vector representation of all the features that have been captured: shapes,

texture, colors, objects, spacial relations, ... etc.

2. Now it's time the model turns those embeddings into real semantic representations, nevertheless, the model still does not know how to label concepts directly. So the model must know combine the embeddings obtained in the image model with the ones from the text model output, aligning them into a shared vector space establishing the real relations and links between the pairs. This bridging mechanism between the image and text pair is where the CLIP model's magic really shines. Although CLIP uses a contrastive alignment strategy in order to put correct pairs closer and dissimilar pairs further in the vector space, there are other models such as *LlaVa* use other tools such as projection layers, where a small network transforms embeddings into LLM token space.
3. Once the image is alligned to the text, that pair is passed to a large language model and then that same LLM takes the combined embeddings and creates meaningful text.
4. Finally, those LLM outputs are transformed into descriptions or answers depending on the task: classification as in the referenced schema image or other objectives such as visual question answering.

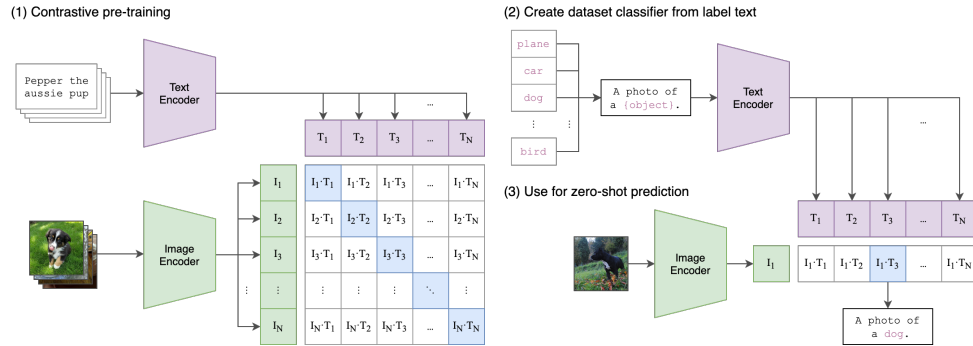


Figure 2.2: CLIP workflow

The reason why these models work so well is because of the millions of image-text pair instances used for training and due to the model's ability to learn and deal with statistical spatial correlations that are shown both in the images and texts.

The dataset the model was trained on is a *WebImageText* (WIT) containing more than 400 million pair instances scraped from the internet, almost half a million text queries and more than 20,000 image-text pairs as query. All of those queries were generated by the most frequent words of the english Wikipedia and then extended using bigrams. The dataset remains private and has never been released.

Last but not least, images are preprocessed by dividing them into the RGB (red, green and blue) channels and normalizing the values so they fall between range 0 to 1. Finally those values are merged with the mean and standard deviation of the images from the training dataset. In case the input image does not have the desired resolution (224x224 usually) the image is scaled by *bicubic interpolation* [6] to the shorter sides is the same as the native resolution, and then the central square of the image is cropped out.

2.1.1.4 Models released in the last years

Since 2022 plenty of new VLM architectures have been published based on similar design and architectures, models such as *Flamingo*, *LLaVA*, *InstructBLIP*, ... could be mentioned. All of these models used a separately trained CLIP-like image encoder standard LLMs for text encoding. However, the release of *GPT-4V* in 2023 marked the emergence of commercial implications, followed by the integration of VLMs into the most powerful models such as *Gemini*, *Copilot* and *Claude 3 Opus*. These newer applications contained a substantially larger number of parameters, were based on training on larger datasets and were powered by greater computational power.

2.1.2 Elements of a VLMs

Inspired in other previous model architectures, vision language models consisted on two main components: the *language encoder* and the *visual encoder*. Compared to other *seq2seq* architectures like encoder-only, decoder-only or encoder-decoder, VLMs consist on 2 encoders, one for each data modality encoding task.

2.1.2.1 Transformers

Before going in detail of the components, I will go over the basic architecture most of the VLMs are based on: *transformers*.

1. The encoders transform the input sequence into numerical representations called *embeddings* which capture all the semantic and positional meaning of the tokens belonging to the input.
2. Then they use a mechanism called *self-attention* which allows the encoder to focus on the most relevant tokens of the sequence independent of their position.
3. Finally the decoders of the transformers take both the output of the self-attention mechanism and the encoder generated embeddings to generate the most statistically likely output sequence.

2.1.2.2 Language encoder

This first encoder captures the semantic relations and meanings between the words in the textual prompt given and turns them into *text embeddings* to be processed, carrying the same encoding process seen in the majority of architectures.

2.1.2.3 Vision encoder

The second encoder extracts the visual properties and patterns from the visual data of the prompt and transforms them into embeddings too. Compared to earlier VLMs which used *convolutional neural networks* (CNNs) for feature extraction such as the colors, shapes and textures from the input, modern ones employ an architecture called *visual transformer* (ViT).

2.1.2.4 Visual transformer (ViT)

Visual transformers process images into smaller patches and treats them as sequences, implementing also the self-attention mechanism across the patches to create a transformer-based representation of the input message.

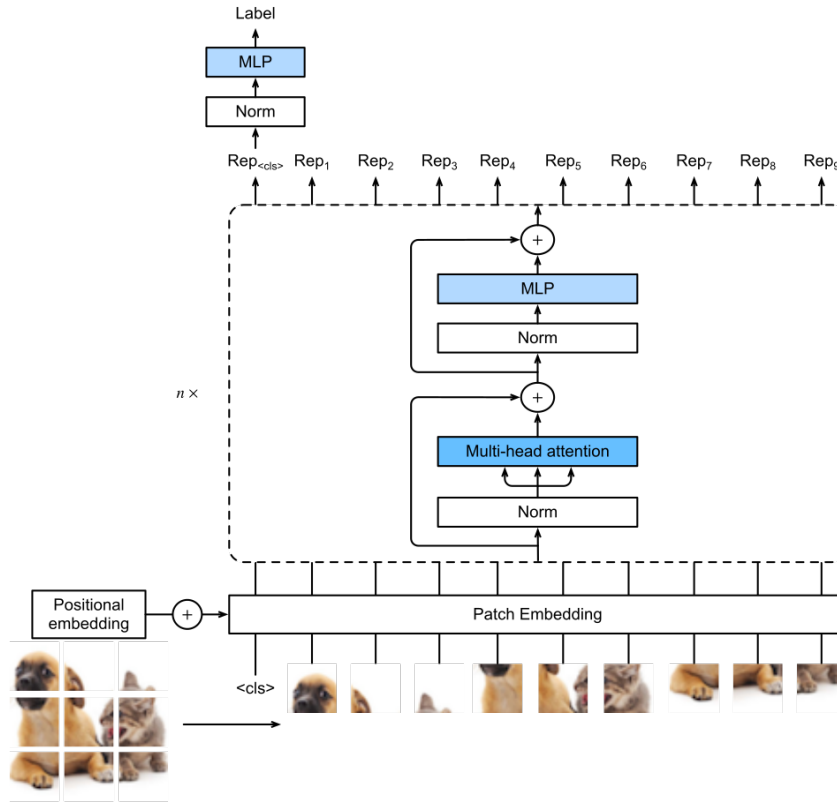


Figure 2.3: Vision transformer architecture

2.2 REINFORCEMENT LEARNING

2.3 MULTI-AGENT SYSTEMS

2.4 STATE OF THE ART

2.4.1 VLMs evolution

2.4.2 Current models

2.4.3 Benchmarks

2.4.4 VLMs in robotics

2.4.5 Frameworks

3. SOLUTION APPROACH

3.1 SOLUTION PROPOSAL

3.2 SYSTEM ARCHITECTURE

3.3 TOOLS USED

3.4 WORKFLOW

4. SOLUTION DEVELOPMENT

4.1 ISAACSIM ENVIRONMENT DEFINITION

4.1.1 Agents

4.1.2 Cameras

4.1.3 Physics

4.2 INDIVIDUAL COMPONENT OPERATION

5. EXPERIMENTATION AND EVOLUTION

5.1 FIRST SCENARIOS

5.2 LAST SCENARIO

5.3 SUCCESSFUL AND FAILURE TRIES

5.4 RESULTS OBTAINED IN EACH SCENARIO

5.5 RESULT DISCUSSION

6. CONCLUSIONS

6.1 CONCLUSIONS AND LESSONS LEARNED

6.2 LIMITATIONS FOUND

The main drawbacks I have found during the development and research process during the whole work have been mainly hardware-related as the specific software requirements necessary to carry on the development are severely demanding due to the fact that the majority of the processes are run and rendered in the GPU. Although I have had access to a good graphical processing unit, the other components of the my working device have fallen behind. Taking for instance the minimum requirements of the used framework *IsaacSim*, I found myself in the limits of the bare minimum, and some of my components such as the CPU were worse than the required. However, I have been able to carry on anyways although there have been a few moments where the I have really felt the bottleneck. When it comes those moments I have felt limited and slowed down due to my equipment are the following:

- I have needed to split some of the working components of the pipeline process inside *IsaacSim* as the whole process would not fit in my GPU's VRAM. The simple fact of opening the framework already consumed close to a quarter of the graphic card's total capacity, and starting the server to listen and interact with the VLM calls alongside left little to no room for more processes in the background. The fact of working on the edge of the minimum hardware requirements also slowed the rendering of visual work and reduced the number of frames per second, nevertheless, this did not obstruct the development process.
- When it comes to making the most of the reinforcement learning training process, I have needed to tweak some of the predefined system configurations to maximize the performance and reduce any kind of lagging and overflow. Adapting the GPU capacity properties to maximize the overall functioning. If I were to work with better equipment, I would have liked to go for longer and deeper trainings, however, I am happy with the performance I obtained when tweaking the parameters I will have discussed in the corresponding section and other strategies such as parallelization.
- Not all limitations have been hardware related, as the official NVIDIA documentation websites were usually corrupted depending on the version. Leading to consulting external pages to solve the mismatching dependencies or installation processes for the frameworks used.

- After consulting the official documentation of the *IsaacLab* repository, I found that even the default reinforcement learning training examples and demonstrations did not work as intended independently of the version. I was not the only one with the same issues as other peoples raised the same issues in different forums, so I inspired in the solutions given by the users with the same problems.

6.3 FUTURE WORK

BIBLIOGRAPHY

- [1] Pietro Bolcato. Understanding vision-language models (vlms): A practical guide, 2024.
- [2] Gravio Team. Vision-language models (VLM) vs visual question answering (VQA) in 2025? <https://www.gravio.com/en-blog/vision-language-models-vlm-vs-visual-question-answering-vqa-in-2025>, Mar 2025. Gravio Blog, Accessed: [Insert Date Here].
- [3] IBM. What are vision-language models? <https://www.ibm.com/think/topics/vision-language-models>, 2024.
- [4] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.
- [5] M. Timothy. What is phrase grounding? <https://blog.roboflow.com/what-is-phrase-grounding/>, Nov 2024. Roboflow Blog, Accessed: [Insert Date Here].
- [6] Wikipedia contributors. Bicubic interpolation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Bicubic_interpolation, 2024. [Online; accessed 19-April-2026].
- [7] Wikipedia contributors. Contrastive language-image pre-training — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Contrastive_Language-Image_Pre-training, 2024. [Online; accessed 19-April-2026].
- [8] Wikipedia contributors. Vision-language model — Wikipedia, the free encyclopedia, 2026.

7. APPENDIX

7.1 INSTALATION GUIDE

IsaacSim/Lab

7.2 RELEVANT CODE

7.3 PROMPTS USED

Prompts used for the VLM

7.4 RESULT TABLE

7.5 TENSORBOARD ANALYSIS